

CHATGPT

¿Qué es el Testing?

El **testing** es el proceso de **probar un software** para verificar que **funcione correctamente**, que **haga lo que se espera** y que **no tenga errores**.

"El testing es una **disciplina del proceso de desarrollo de software** cuya finalidad es verificar y validar que un producto cumpla con los **requisitos especificados**, detectando errores, fallos o desviaciones respecto al comportamiento esperado."

-  **Verificar** que el software hace lo que tiene que hacer (según los requerimientos).
-  **Detectar defectos** antes de que lleguen al usuario final.
-  **Prevenir errores futuros**, entendiendo cómo se comporta el sistema en distintos contextos.
-  **Mejorar la calidad** del producto final (robustez, confiabilidad, usabilidad).
-  **Asegurar la conformidad** con criterios legales, normativos o contractuales (según el tipo de software).

El objetivo principal del testing **no es demostrar que el software funciona**, sino **encontrar los errores que podrían impedir que funcione correctamente**.

Manifiesto Testing

una declaración de principios moderna y ágil que busca **redefinir el rol del testing** dentro del proceso de desarrollo de software.

Manifiesto Testing

1. "Testing throughout over testing at the end"

Probar durante todo el desarrollo en lugar de **esperar al final del proceso**.

- ♦ Tradicionalmente, las pruebas eran una etapa que venía después de la codificación (modelo en cascada).
- ♦ Este manifiesto propone incorporar pruebas **desde el principio**, como parte **continua** del proceso.

2. "Preventing bugs over finding bugs"

Prevenir errores en lugar de **solo encontrarlos**.

- ♦ El testing no debe limitarse a "cazar bugs", sino que debe contribuir desde el diseño a **evitar que ocurran**.

- ◆ Esto se logra participando en reuniones de planificación, revisión de requerimientos, diseño, etc.

3. “Testing understanding over checking functionality”

Probar el entendimiento del sistema en vez de solo verificar funcionalidades.

- ◆ No se trata solo de hacer clics y ver si el botón funciona.
- ◆ El tester debe entender **qué problema se está resolviendo**, para quién y por qué.

4. “Building the best system over breaking the system”

Contribuir a construir un buen sistema más que intentar romperlo.

- ◆ El tester no es un "saboteador", sino un **colaborador activo** en la mejora del producto.
- ◆ La mentalidad no es “a ver qué falla”, sino **“cómo aseguramos que esto le sirva al usuario”**.

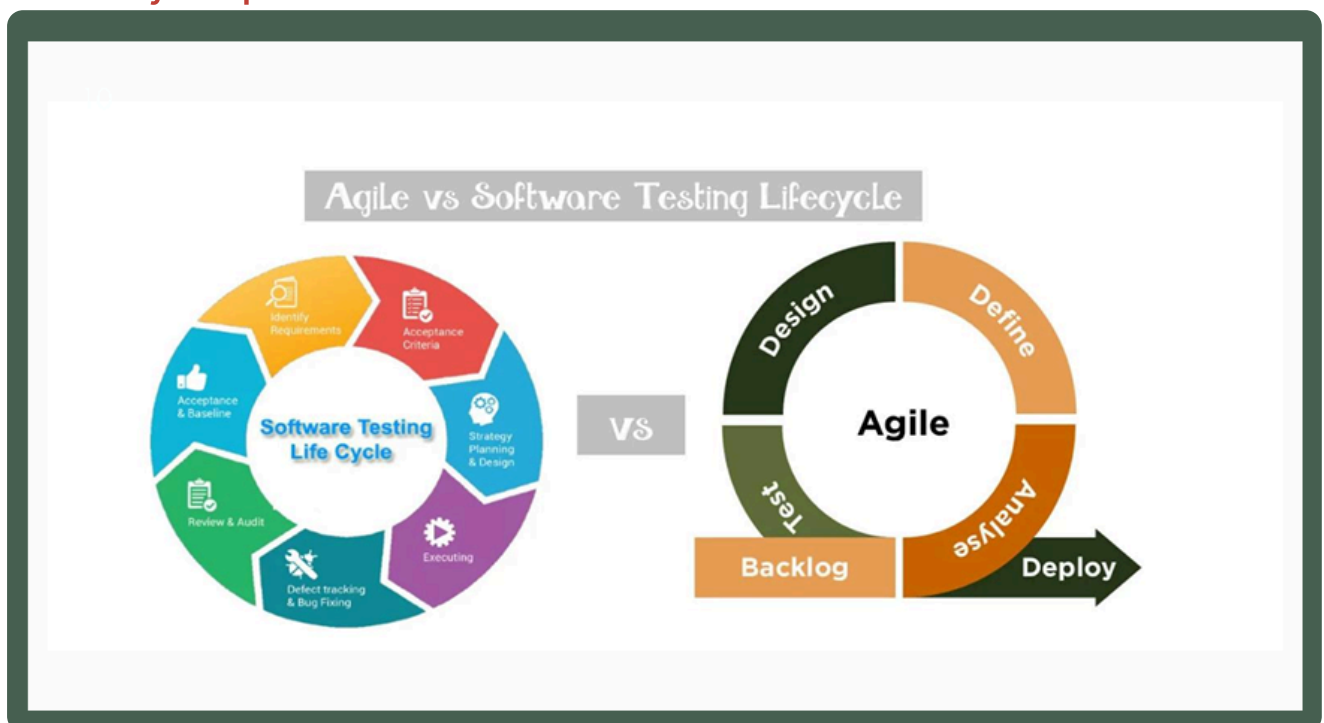
5. “Team responsibility for quality over tester responsibility”

La calidad es responsabilidad del equipo, no solo del tester.

- ◆ En enfoques ágiles, **la calidad es compartida**: programadores, analistas, testers y PO trabajan juntos.
- ◆ El tester no es "el que prueba al final" sino un **facilitador de calidad** durante todo el desarrollo.

Agile vs Software Testing Life Cycle (STLC)

Ambos modelos se relacionan con el desarrollo de software, pero **tienen enfoques distintos y complementarios**:



A la izquierda: **Software Testing Life Cycle (STLC)**

Se trata del **ciclo de vida del testing**, un proceso formal y estructurado que define **cómo se realizan las pruebas de software** de manera sistemática.

Fases del STLC:

1. **Identify Requirements (Identificar Requisitos)**
Comprender qué se va a probar. Aquí el equipo de testing analiza la documentación del sistema (user stories, casos de uso, requisitos).
2. **Acceptance Criteria (Criterios de Aceptación)**
Se definen las condiciones que debe cumplir el software para ser considerado aceptable. Suele incluir criterios funcionales y no funcionales.
3. **Strategy Planning & Design (Planificación y diseño)**
Se planifican las pruebas: qué tipo, cuántas, con qué herramientas, tiempos, cobertura, ambientes, etc.
4. **Executing (Ejecución)**
Se ejecutan los casos de prueba diseñados, verificando el comportamiento real vs. esperado.
5. **Defect Tracking & Bug Fixing (Seguimiento de errores y corrección)**
Se documentan los errores encontrados y se trabaja junto al equipo de desarrollo para resolverlos.
6. **Review & Audit (Revisión y auditoría)**
Se revisa la calidad del proceso de pruebas y del producto probado. Puede incluir métricas, reportes, retroalimentación.
7. **Acceptance & Baseline (Aceptación y línea base)**
Si las pruebas son satisfactorias, el software se aprueba y se establece como base para futuras versiones.

A la derecha: **Ciclo Ágil de Desarrollo**

Representa el **proceso iterativo e incremental** de desarrollo de software bajo una metodología ágil (como Scrum, XP, etc.).

Fases del ciclo ágil:

1. **Define**
Se definen los requerimientos funcionales (en forma de user stories o features).
2. **Analyse**
El equipo analiza cómo implementar esas funcionalidades: dependencias, esfuerzo, impacto, etc.
3. **Deploy**
Se lanza una versión funcional del software (a staging o producción).
4. **Test**
Se prueban las funcionalidades, idealmente en paralelo al desarrollo (testing continuo o automatizado).

5. Backlog

Se mantiene una lista priorizada de tareas pendientes. Alimenta nuevas iteraciones.

6. Design

Se diseña la solución de forma colaborativa y adaptable.

- El STLC es como el **“manual tradicional” del tester**.
- El Agile es como el **nuevo enfoque colaborativo y ágil** de desarrollo.
- **No compiten**, sino que **pueden convivir** si se aplican correctamente.

Característica	Cascada (Tradicional)	Ágil
Proceso	Secuencial y rígido	Iterativo e incremental
Orden de fases	Lineal: Req → Diseño → Dev → Test	Paralelo: todas las actividades en cada Sprint
Involucramiento del cliente	Al inicio y al final	Durante todo el proceso
Entrega de software	Al final del proyecto	En cada iteración (frecuente)
Cambio de requerimientos	Costoso y difícil	Bienvenido y frecuente
Testing	Tardío (después del desarrollo)	Temprano y continuo
Documentación	Detallada y formal	Ligeramente documentada, suficiente
Riesgo de fallar	Alto si hay errores tarde	Bajo: se corrige continuamente

9 principios clave para aplicar Testing en proyectos Ágiles

1. Testing se mueve hacia adelante en el proyecto

Las pruebas no se dejan para el final del desarrollo, sino que se **planifican desde el inicio** del Sprint.

- Detectar errores **temprano**.
- Reducir el costo de los fallos.
- Incluir criterios de aceptación desde la redacción de user stories.

2. Testing no es una fase

Testing **no es una etapa separada** o posterior al desarrollo. Es una **actividad continua e integrada**.

- Elimina el enfoque secuencial del modelo en cascada.
 - Favorece la **entrega continua** y la integración constante.
 - Permite que el código se valide mientras se desarrolla.
-

3. Todos hacen testing

No solo los testers prueban. **Desarrolladores, analistas, PO** y demás roles son responsables de la calidad.

- Fomentar la **responsabilidad compartida** por la calidad.
 - Facilita prácticas como **TDD** o revisión cruzada.
 - Mejora la colaboración y reduce errores de comunicación.
-

4. Reducir la latencia del feedback

Obtener respuestas rápidas sobre si algo funciona o no. Idealmente en **minutos, no días**.

- Detectar errores rápido = corregir rápido.
 - Permite ciclos de mejora continua (Build → Test → Fix).
 - Automatización juega un rol clave acá.
-

5. Las pruebas representan expectativas

Cada prueba debe reflejar lo que el usuario espera que el sistema haga (funcionalidad y valor).

- Asegura que el producto **cumpla los requerimientos reales**.
 - Las pruebas se diseñan junto con el PO o usuario (por ejemplo, con criterios de aceptación en User Stories).
-

6. Mantener el código limpio, corregir los defectos rápido

El equipo debe aplicar buenas prácticas (clean code, refactorización) y **resolver bugs al instante**.

- Evita acumulación de “deuda técnica”.
- Mejora la mantenibilidad del software.

- Permite entregas más estables y sostenibles.
-

7. Reducir la sobrecarga de documentación de las pruebas

Evitar documentos extensos y burocráticos. Se prefiere **automatización y colaboración** en lugar de papeles.

- Aumentar la eficiencia.
 - Mantener documentación **viva** y útil (por ejemplo, pruebas automatizadas como “documentación ejecutable”).
-

8. Las pruebas son parte del “done”

Una historia o tarea **no está terminada hasta que fue probada y pasó los tests**.

- Asegura calidad en cada entrega.
 - Crea una definición clara y objetiva de “hecho”.
 - Vincula calidad con entregas, no solo con código terminado.
-

9. De probar al final a Conducido por Pruebas (Testing-Driven)

Cambiar el paradigma: ahora las pruebas **guían el desarrollo**, no lo validan después.

- Implementar técnicas como **TDD (Test Driven Development)**.
 - Facilita diseño más robusto y orientado al comportamiento.
 - Aumenta confianza en el software desde el primer momento.
-

Introducción a las 6 Prácticas Concretas de Testing en Ágil

Estas **6 prácticas concretas** representan enfoques modernos y efectivos para integrar el testing dentro del desarrollo ágil de software. Se enfocan en la **automatización, colaboración, calidad continua** y adaptabilidad, pilares esenciales en entornos como Scrum, XP o DevOps.

Pruebas de unidad e integración automatizadas

Se trata de **verificar el funcionamiento correcto de componentes individuales (unit)** y su interacción entre sí (integration). Se automatizan para ejecutarse frecuentemente y detectar errores temprano.

Pruebas de regresión a nivel de sistema automatizadas

Aseguran que **nuevos cambios no rompan funcionalidades existentes**. Se ejecutan automáticamente tras cada cambio importante para mantener la estabilidad del sistema completo.

Pruebas exploratorias

Son pruebas **no planificadas paso a paso**, donde el tester usa su experiencia e intuición para **descubrir errores inesperados**. Muy valiosas para detectar fallos fuera de los “camino felices”.

TDD (Test Driven Development)

Desarrollo guiado por pruebas: primero se escribe una prueba (fallida), luego el código mínimo necesario para pasarla y se refactoriza. Asegura que el código esté **pensado para ser probado** desde el inicio.

ATDD (Acceptance Test Driven Development)

Escribir primero las **pruebas de aceptación** en colaboración con el cliente o PO. Sirven como **contrato funcional** y guían el desarrollo según lo que el negocio espera que funcione.

Control de versión de las pruebas con el código

Las pruebas automatizadas (unitarias, integración, regresión, etc.) se almacenan en el **mismo repositorio** que el código fuente. Esto permite versionar, mantener y ejecutar todo en conjunto.



El rol del **Tester** en los equipos ágiles

En los equipos ágiles, **el tester ya no es un verificador al final del proceso**, sino un **actor clave y activo durante todo el ciclo de desarrollo**. Esta imagen representa su función central dentro del equipo y cómo colabora con los distintos roles: **Product Owner**, **Developers** y **System Team**.

Vamos a desglosar lo que muestra la imagen y explicarlo de forma clara y completa.

El Tester como **nodo central de colaboración**

El tester se ubica **en el centro del equipo**, colaborando con:

-  **Product Owner**
 -  **Developers**
 -  **System Team (Equipo de soporte técnico, integración, infraestructura, QA extendido, etc.)**
-

Relación con el **Product Owner (PO)**

El tester trabaja codo a codo con el PO en tareas de **definición funcional y validación**. Lo ayuda en:

- ◆ **Preparación y revisión de historias**
 - Revisa que las historias estén claras y entendibles para ser implementadas y probadas.
 - ◆ **Definición de criterios de aceptación**
 - Participa en definir **qué condiciones debe cumplir una historia para considerarse “hecha” (done)**.
 - ◆ **Entendimiento del dominio del negocio**
 - Aprende junto al PO el problema real que se quiere resolver.
 - ◆ **Identificación de dependencias**
 - Ayuda a detectar interacciones entre historias o sistemas que pueden generar errores ocultos.
-

Relación con los **Developers**

Con los desarrolladores, el tester colabora para garantizar que **el código sea entregado con calidad desde el principio**:

- ◆ **Estrategias de prueba**

- Propone cómo, cuándo y qué probar (unitarias, automatizadas, exploratorias, etc.).

- ◆ **Automatización de pruebas funcionales (ATDD)**

- Implementa pruebas automatizadas que validan el comportamiento del sistema.

- ◆ **Feedback temprano**

- Provee retroalimentación inmediata sobre el estado de las historias en desarrollo.

- ◆ **Sugerencias de mejora**

- Identifica problemas de usabilidad y hace sugerencias funcionales basadas en la experiencia de prueba.

✂ Relación con el **System Team**

Este equipo suele incluir perfiles técnicos enfocados en la **integración continua, ambientes, configuración y despliegue**.

- ◆ **Preparación de documentación para pruebas**

- Informa qué historias se deben integrar y probar.

- ◆ **Difusión del plan de pruebas**

- Comparte qué pruebas se harán, en qué entorno y con qué datos.

- ◆ **Asegura la completitud del feature**

- Verifica que el sistema esté realmente listo para ser entregado, no solo "aparentemente funcional".

- ◆ **Comunicación de bugs**

- Reporta errores y asegura que se comuniquen **a PO y Developers**, manteniendo fluidez.
-

Responsabilidades propias del **Tester** en el equipo

Además de colaborar, tiene sus propias tareas clave:

✓ Preparar y mantener ambientes de prueba

Ambientes controlados para ejecutar casos del sprint.

✓ Validar el “Done”

Confirma que cada historia cumpla todos los criterios definidos.

✓ Registrar bugs

Identifica y reporta errores de forma clara y priorizada.

✓ Medir la calidad

Define métricas (como cobertura, tasa de errores, velocidad de corrección).

✓ Aceptar historias con el PO

Ambos validan si una funcionalidad puede entregarse.

Desafíos culturales en el Testing (Cultural Challenges)

Esta imagen representa los principales **desafíos culturales** que enfrentan los equipos cuando se busca adoptar nuevas prácticas de testing, especialmente en entornos ágiles. Aunque los aspectos técnicos del testing son importantes, **los cambios culturales y de mentalidad son muchas veces el mayor obstáculo**.

Vamos a desglosarlos por categorías:

“Change Doesn’t Come Easy” – El cambio no es fácil

Implementar buenas prácticas de testing **requiere cambiar la forma en que los equipos piensan y trabajan**, y eso no es inmediato.

Subtemas:

- **Be patient:** la transformación lleva tiempo.
- **Let them feel the pain:** que el equipo vea los problemas actuales ayuda a valorar el cambio.
- **Build your credibility:** ganarse la confianza del equipo técnico para poder influir.
- **Professional development:** invertir en la capacitación continua del equipo.

- **Beware the quality police mentality:** no actuar como “la policía de calidad”; el tester es guía, no fiscalizador.
 - **Vote with your feet:** si el entorno es muy tóxico y no hay voluntad de mejora, a veces la mejor acción es salir de allí.
-

Organizational Culture – Cultura organizacional

Cada empresa tiene valores, estructuras y prácticas que pueden **favorecer o bloquear** la incorporación del testing como parte fundamental del desarrollo.

Subtemas:

- **Quality philosophy:** ¿La organización cree que la calidad es importante?
 - **Sustainable pace:** trabajar a ritmo saludable favorece testing efectivo.
 - **Customer relationships:** si se trabaja cerca del usuario, es más fácil validar valor.
 - **Organization size:** en organizaciones grandes puede haber más resistencia al cambio.
 - **Empower your team:** los equipos deben tener autonomía para decidir cómo garantizar la calidad.
-

Barriers to Success – Barreras al éxito

Existen **barreras personales, técnicas y organizacionales** que dificultan adoptar prácticas modernas de testing.

Subtemas:

- **Loss of identity:** algunos roles (como testers tradicionales) pueden sentir que pierden su lugar.
 - **Additional roles:** al haber más tareas compartidas, algunos pueden resistirse.
 - **Lack of training:** falta de formación técnica y metodológica.
 - **Not understanding Agile concepts:** si no se entienden los valores ágiles, no se entiende el rol del testing ágil.
 - **Past experiences:** malas experiencias previas pueden generar rechazo.
 - **Cultural differences among roles:** distintos perfiles (PO, devs, testers) pueden tener formas diferentes de pensar.
-

Management Expectations – Expectativas de gestión

Muchas veces los líderes no comprenden lo que implica cambiar el enfoque de calidad.

Subtemas:

- **Cultural change for managers:** los líderes también deben cambiar su mentalidad.
- **Speaking the manager's language:** los testers deben aprender a comunicar el valor del testing en términos de negocio.

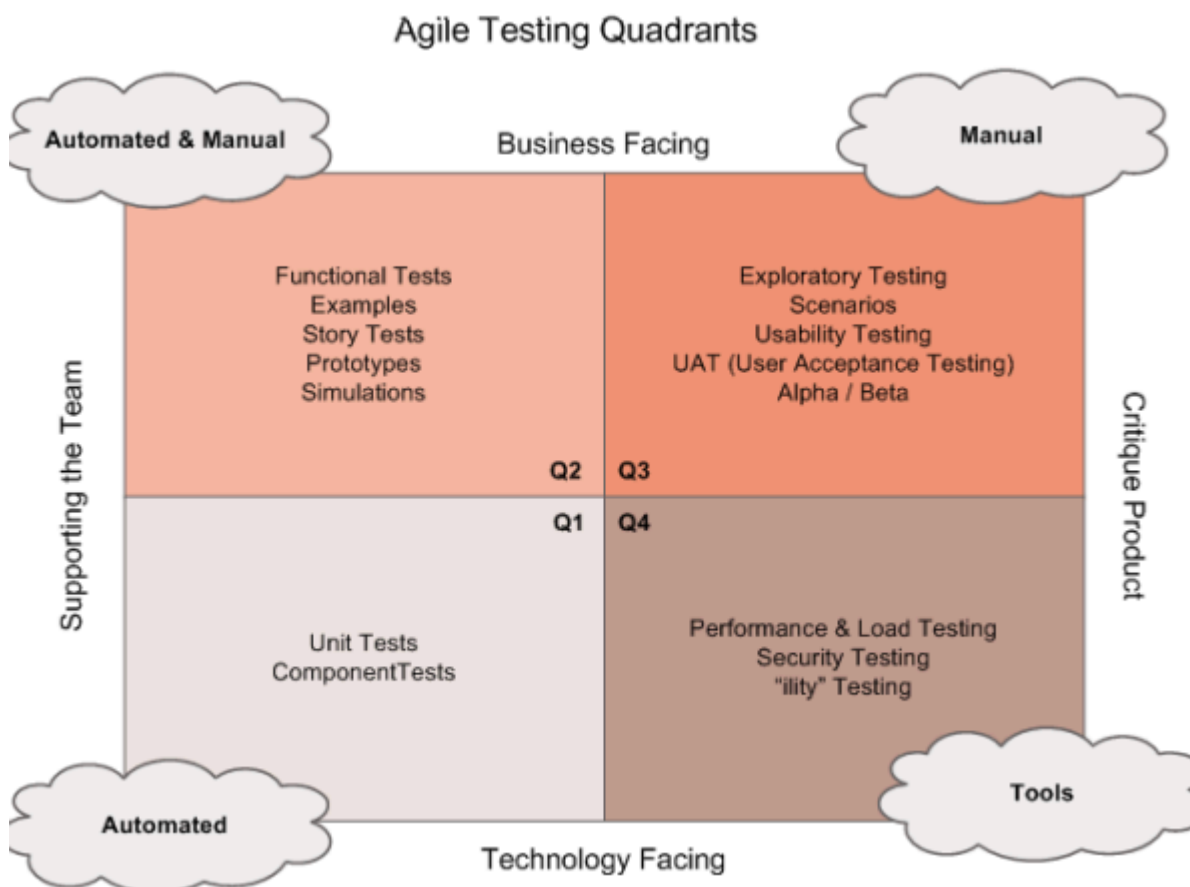
Introducing Change – Introducir el cambio

Cambiar la cultura de testing **requiere estrategia, empatía y participación del equipo.**

Subtemas:

- **Talk about fears:** abrir espacios para hablar de miedos y resistencias.
- **Give team ownership:** dar al equipo el control y responsabilidad sobre la calidad.
- **Celebrate success:** reconocer logros incrementales para mantener motivación.

Cuadrante del testing ágil



Piramide

La **pirámide de testing ágil** es un modelo visual y conceptual que ayuda a organizar las pruebas según su nivel, tipo y propósito. Su objetivo es **lograr una estrategia de**

pruebas eficiente, escalable y sostenible, que acompañe el desarrollo ágil de software.

- **Prioriza el testing automatizado y temprano.**
- **Evita la sobredependencia de pruebas manuales de UI (lentas y frágiles).**
- **Reduce el costo y tiempo de detección de bugs.**
- **Asegura calidad desde la base técnica hasta la experiencia del usuario.**



◆ **1. Pruebas Unitarias / de Componentes**

✦ **¿Qué son?**

- Pruebas hechas por los programadores.
- Verifican unidades de código de forma aislada (funciones, clases, métodos).

⚙ **Automatizadas.**

🎯 **Objetivo:** detectar errores en lo más pequeño antes de integrar.

📖 **“Drive Development”:** Son la base del desarrollo guiado por pruebas (TDD).

◆ 2. Pruebas Funcionales / Reglas de Negocio

✦ ¿Qué son?

- Validan que el sistema cumpla con la lógica de negocio esperada.

🔧 Ej.: ¿Se calcula bien un descuento? ¿Se aplica bien una validación?

⚙️ Automatizadas, especialmente a través de **API o servicios**.

💬 Se centran en lo que **el sistema debe hacer**, no cómo se ve.

◆ 3. Pruebas de Flujo de Trabajo a través de la Interfaz Gráfica (GUI)

✦ ¿Qué son?

- Simulan el uso del sistema como lo haría el usuario final (navegación, clicks, formularios).

📊 Más frágiles y lentas.

💻 Pueden ser automatizadas (con herramientas como Selenium), pero muchas veces son **manuales**.

● En Ágil deben ser las menos numerosas (1-5%), debido a su costo y fragilidad.

◆ 4. Pruebas Manuales / Exploratorias

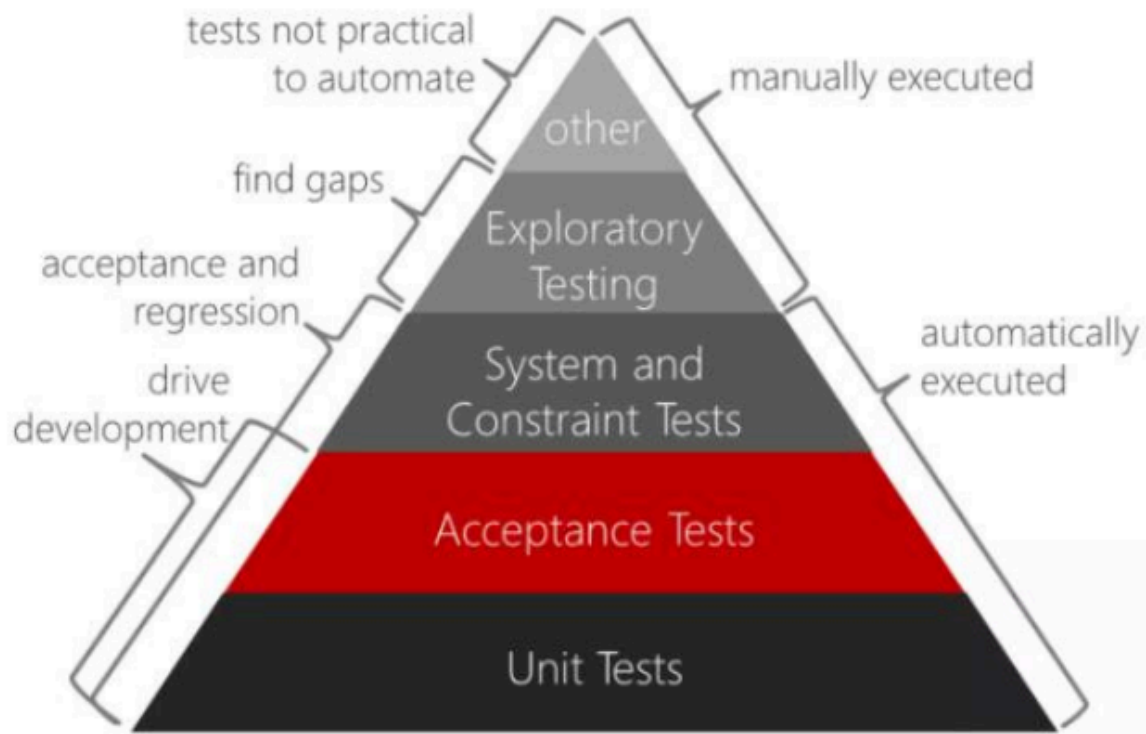
✦ ¿Qué son?

- No siguen scripts rígidos.
- El tester usa su intuición y conocimiento del dominio para **explorar** el sistema y buscar fallos ocultos.

💬 Muy útiles para:

- Encontrar gaps (“**find gaps**”) que no cubren las pruebas automatizadas.
- Evaluar usabilidad, experiencia, percepción.

⊘ “**Tests not practical to automate**”: este tipo de testing no siempre se puede automatizar, por eso permanece **en la parte superior de la pirámide**.



-  **Drive Development:** Unit tests ayudan a diseñar buen código.
-  **Acceptance and Regression:** Acceptance tests permiten probar funcionalidades y asegurar que no se rompa lo ya hecho.
-  **Find Gaps:** Testing exploratorio descubre lo no previsto.
-  **Tests not practical to automate:** Manuales, cuando la automatización no vale la pena.

Aspecto	Testing Tradicional	Testing Ágil
Foco principal	Detectar errores (post-desarrollo)	Prevenir errores (desde el diseño)
GUI Tests	80–90% del esfuerzo	1–5% del esfuerzo
Unit Tests	1–5%	80–90%
Velocidad	Lento, manual	Rápido, automatizado
Costo	Alto (bug tardíos)	Bajo (bug temprano)
Feedback	Tardío	Inmediato (continuous feedback)

Beneficios de la automatización del testing

Rápida
ejecución

Reusabilidad

Mayor
cobertura

Mayor
precisión

Mayor
alcance

Reducción
de costos

TDD

“El acto de diseñar tests es uno de los mecanismos conocidos más efectivos para prevenir errores...El proceso mental que debe desarrollarse para crear tests útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo”

Leyes TDD

- No escribir una línea de código hasta que no hayas escrito antes un test case que falla
- No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente.
- No vas a escribir más código en Producción que el necesario para que el test pase.

Aplicar estas leyes no significa no aplicar test, el test se aplica siempre...

Escribir test

Hacer que test compile
que falle

Cambio para que el test pase

Corre el test para que pase

Quitar duplicaciones (refactor)

Patrones de TDD

(Tengo fotos en el celular)

utilizar la palabra test

armar sets de test

primero escribo el resultado de la prueba, despues la prueba

datos claros para la prueba

refactorizar

(imagenes en el celular)

concepto en general:

Sobre codigo ya escrito. no corrijo, sino, que emprolijo el codigo, solo que sea mas facil de leer, mantener, cumpla prinncipios de diseño. quito duplicidad de codigo.

como hacer refactoring

(cel)

asegurarse que las pruebas pasen

encontrar un codigo que huela mal

determinar como simplificar el codigo

hacer las simplificaciones

...

Resumen



ver video

test driven development

aprende a triangular en tdd

<https://www.youtube.com/watch?v=1ggtkO9JKtU>

practico TDD

Vemos el ejercicio de practico taxi mobile

ver los assert, es una de las partes mas importantes.

1. Precondiciones
2. Pasos del caso de prueba
3. Resultados
 - assert
 - assert all/any
4. message final

TDD es prueba unitaria

ver diferencia con ATDD

→ ChatGPT

TDD

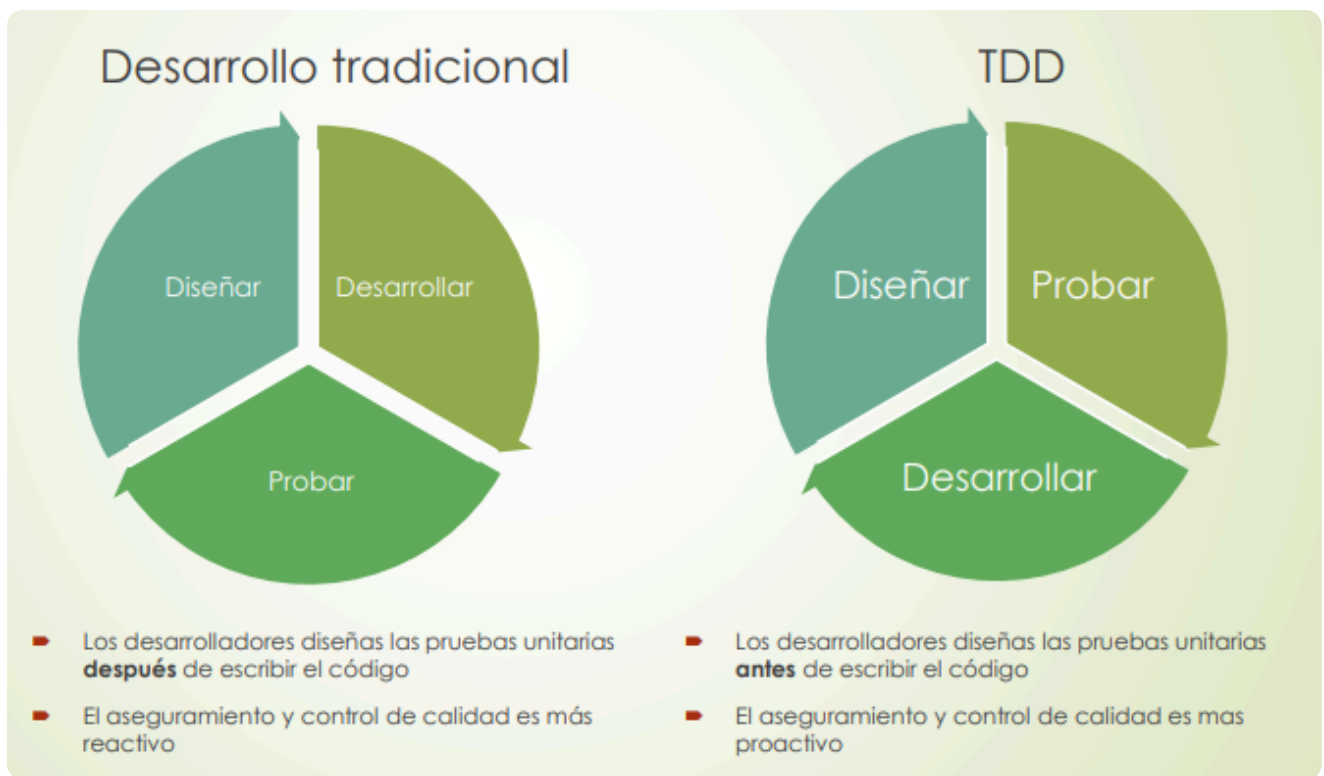
TDD (Desarrollo Guiado por Pruebas) es una técnica de desarrollo de software donde el programador **escribe primero las pruebas** antes de escribir el código que las hace pasar. Es una práctica central del desarrollo ágil, especialmente en metodologías como **Extreme Programming (XP)**.

"Test-Driven Development es un diseño de software mediante pequeños ciclos de desarrollo donde se escribe una prueba que falla, se hace que pase y luego se refactoriza."

- Para **asegurar calidad desde el inicio** del desarrollo.
- Para **diseñar software más robusto, mantenible y simple**.
- Para prevenir errores en lugar de detectarlos después.
- Para **validar que el código cumple con los requerimientos funcionales**.
- Para **facilitar el cambio y el refactor**, manteniendo la confianza en que no se rompe nada.

TDD sigue el ciclo conocido como **Red – Green – Refactor**:

1.  **Red**: Escribir una prueba **que falle** (porque el código aún no existe).
2.  **Green**: Escribir el código mínimo necesario **para que la prueba pase**.
3.  **Refactor**: Mejorar el código manteniendo las pruebas verdes (pasan).



Proceso	¿Cuándo se prueba?	¿Qué implica?
Tradicional	Después del desarrollo	Control de calidad reactivo
TDD	Antes de codificar	Calidad proactiva desde el diseño

🔍 En TDD **el diseño nace de la necesidad de pasar una prueba**, por eso se dice que es un enfoque **de diseño emergente**.

Es una técnica avanzada que involucra otras dos prácticas: Escribir las pruebas primero (Test First Development) y Refactorización (Refactoring).

Para escribir las pruebas generalmente se utilizan las pruebas unitarias

Beneficios

Código robusto

- 📌 Código más predecible y limpio, que permite saber cuándo el código está terminado
- 📌 Código más tolerable al cambio, al tener escritas las pruebas es más fácil de entender
- 📌 Código más seguro
- 📌 Aumenta la posibilidad de reducir la duplicación de código (si el test pasa el código ya existe)
- 📌 Código más barato de mantener, al ser más entendible
- 📌 Con la práctica acelera la velocidad de desarrollo y la posibilidad de despliegue continuo
- 📌 Asegura cobertura de prueba, lo que conlleva a código de calidad
- 📌 Obliga a pensar en cómo usar el componente, por lo tanto, el diseño de las API
- 📌 Promueve el desarrollo de componentes con alta cohesión y bajo acoplamiento

¿Dónde se utiliza TDD?

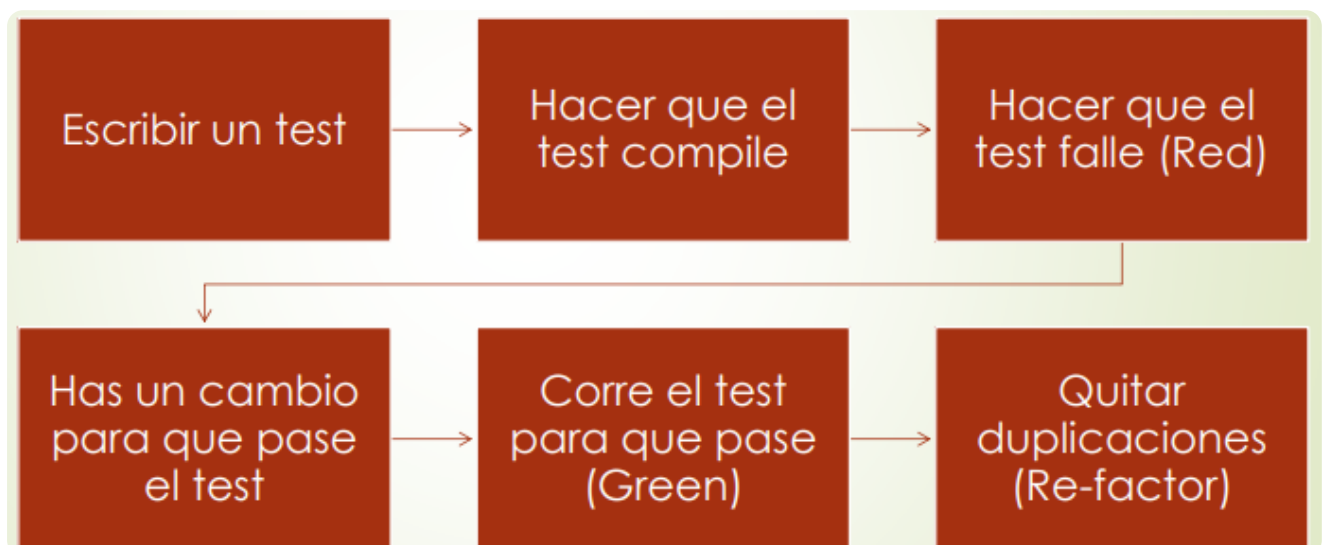
- En proyectos ágiles con foco en calidad desde el comienzo.
- En desarrollo de aplicaciones donde la **seguridad, fiabilidad o mantenibilidad** son claves.
- Con tecnologías y frameworks que soporten testing automatizado (JUnit, Jest, PyTest, etc.).
- En **código nuevo**, donde se puede diseñar desde cero bajo esta lógica.

Aunque su nombre diga "Test", **TDD es una práctica de diseño**. Las pruebas son solo el vehículo que **obliga al desarrollador a pensar primero en el comportamiento deseado**, y luego en la implementación.

Elemento	TDD	Tradicional
¿Cuándo se escriben las pruebas?	Antes de codificar	Después de codificar
¿Quién las escribe?	El desarrollador	El tester u otro desarrollador
¿Cómo se diseña el software?	Basado en requerimientos expresados como tests	Basado en documentos o modelos formales
Tipo de aseguramiento	Preventivo (proactivo)	Correctivo (reactivo)

LEYES TDD

- No escribir una línea de código hasta que no hayas escrito antes un test case que falla. No es necesario escribir más test de los necesarios para que falle.
- Normalmente con un test que falle es suficiente.
- No vas a escribir más código en Producción que el necesario para que el test pase.



Patrones TDD

Patrones red bar

- One Step Test: Escribe un solo test que verifique una pieza mínima de funcionalidad.
- Starter Test: Primer test que te ayuda a explorar la API o comportamiento deseado.
- Explanation test: Escribe comentarios o preguntas para aclarar qué esperas que haga el código
- Learning test: permite conocer APIs externas sobre las que no sabemos su comportamiento.
- Another Test: Añade un nuevo test para cubrir un caso adicional o una variante del comportamiento.
- Regression Test: Agrega un test que reproduce un bug descubierto, para evitar regresiones futuras.

Patrones green bar

- Fake it: Devuelve un valor fijo o solución "falsa" solo para hacer pasar el test.
- Triangulación: Introduces un **segundo test con diferentes datos** para forzar una implementación más general
- Obvious implementation: Cuando la solución es trivial y obvia, puedes implementarla directamente sin "Fake It".
- One to many: Patrón para resolver test para colecciones de elementos.

Patrones de TDD

- Test: para nombrar las pruebas con la palabra test y la clase a probar.
- Isolated test: mantener los test totalmente independientes
- Test List: primero escribe la lista de tests que vas a realizar
- Test first: escribe primero la prueba
- Assert test: primero escribir la confirmación y luego la prueba
- Test data: usa datos para el testing que sean fáciles de leer y seguir.
- Evident data: Incluye los resultados esperados y reales dentro del test, e intenta que la relación entre los distintos datos sea evidente.

Otros patrones

- Patrones de testing
- Patrones de diseño
- xUnit Patterns

¿Qué es el Refactoring?

El **refactoring** (o refactorización) es una técnica de desarrollo de software que consiste en **modificar el código fuente para mejorar su estructura interna sin cambiar su comportamiento externo**.

El objetivo principal del refactoring es **hacer el código más limpio, entendible, mantenible y flexible**, sin alterar lo que el software hace.

- ✂ **Eliminar duplicación o complejidad innecesaria**
- 📖 **Mejorar la legibilidad** y comprensión del código
- 🔄 **Flexibilizar y modularizar** para permitir cambios futuros
- ✅ **Preservar el comportamiento del sistema** (todas las pruebas siguen pasando)

- 🗨️ **Incorporar lo aprendido** sobre el dominio y mejorar el diseño progresivamente

El **Refactoring** es la tercera fase del ciclo TDD:

1. 🛑 **Red**: Escribo una prueba que falla.
2. ✅ **Green**: Escribo el código más simple para pasar la prueba.
3. ♻️ **Refactor**: Reorganizo y mejoro el código sin romper la funcionalidad.

En otras palabras, TDD nos permite **refactorizar con confianza**, porque las pruebas automatizadas que ya escribimos garantizan que no rompemos nada.

¿Qué se hace durante el refactoring?

- ✅ **Verificar que las pruebas pasen** (esto garantiza que partimos de un estado funcional).
- 🚨 **Identificar "code smells"** (malos olores: código duplicado, funciones largas, nombres malos, etc.)
- 🗨️ **Analizar cómo simplificar o mejorar el código.**
- ✂️ **Aplicar mejoras** (renombrar variables, dividir funciones, eliminar redundancia...).
- ✅ **Volver a ejecutar las pruebas** para confirmar que el comportamiento no cambió.

Mejora de estructura interna, preservando la externa

- **Estructura externa**: lo que el sistema hace, las funciones visibles para el usuario final o cliente (se mantiene igual).
- **Estructura interna**: cómo está escrito, organizado y mantenido el código (es lo que se mejora).

✂️ **Refactoring NO agrega funcionalidades nuevas**, simplemente **hace mejor lo que ya existe**.

Refactoring y pruebas

Las pruebas son **fundamentales** para el refactoring porque:

- Dan **confianza** al desarrollador para hacer cambios sin miedo a romper cosas.
- Validan que, aunque el código cambió, el sistema **sigue comportándose igual**.
- Por eso, se dice que el refactoring **necesita un buen soporte de pruebas automatizadas** (como las que provee TDD).

📝 Nota

- Evita el deterioro del diseño ("code rot")

- Limpia desorden y simplifica
- Incrementa legibilidad y comprensión
- Facilita detectar errores
- Reduce el tiempo de depuración
- Permite incorporar aprendizajes
- Forma parte natural de todo proceso creativo

☀ Resumen

✅ ¿Qué es TDD?

TDD (Test-Driven Development) es una **técnica de desarrollo** que consiste en: escribir primero una **prueba que falla** (test), luego el **código mínimo** para que pase, y finalmente **refactorizar** para mejorar el diseño.

📖 *“Test-driven development es un proceso de desarrollo donde se escriben pruebas antes del código funcional. Estas pruebas guían el diseño del software.”*

— **Kent Beck, Test-Driven Development: By Example**

🔍 No es una **metodología** ni un **framework**. 💡 Es una **técnica** usada dentro de metodologías ágiles (XP, Scrum), aplicada a nivel de programación.

⚙ Las 3 fases del ciclo TDD

- **Red**: Escribir una prueba automatizada que **falle** (la funcionalidad aún no existe).
- **Green**: Escribir el **mínimo código** para que la prueba pase.
- **Refactor**: **Mejorar el código** (legibilidad, diseño, duplicación) sin cambiar su comportamiento.

Este ciclo se repite muchas veces al día.

🔄 Refactoring en TDD

- Es el **tercer paso** del ciclo.
- Mejora el **diseño interno** sin cambiar lo que hace.
- Se realiza **con todas las pruebas en verde**.

Beneficios: elimina duplicación/complicidad innecesaria, aumenta **claridad, mantenibilidad y flexibilidad**, y **reduce errores futuros**.

🌀 Las 3 Leyes del TDD (Robert C. Martin)

1.  No escribir **código funcional** sin antes tener una **prueba que falle**.
2.  No escribir **más prueba** de la necesaria para que **falle**.
3.  No escribir **más código** del necesario para **pasar la prueba**.

Patrones de TDD: RedBar / GreenBar

-  **RedBar**: comenzar por un **test que falle**; fuerza a pensar el diseño desde el **uso (API)**.
-  **GreenBar**: implementar **lo mínimo** para que el test pase (puede ser “provisional”).
-  Luego **refactor** con la seguridad de los tests automatizados.

TDD vs. Desarrollo tradicional

Aspecto	Desarrollo Tradicional	TDD
Orden de escritura	Código primero, test después (si hay)	Test primero , luego código
Calidad del diseño	Reactiva	Proactiva
Detección de errores	Tarde (testing al final)	Temprano (al escribir código)
Facilidad para refactor	Baja	Alta (tests aseguran cambios)
Cambios de requerimientos	Más costosos	Más controlados y adaptables

¿Para qué se usa TDD?

- Para **guiar el diseño** del software.
- Para **detectar errores temprano**.
- Para lograr **código testeable, desacoplado y flexible**.
- Para **mejorar la calidad** y **reducir el miedo** a cambiar/refactorizar.

¿Dónde se utiliza?

- Entornos ágiles, especialmente **XP**.
- Equipos con **CI/CD**.
- Proyectos con foco en **calidad técnica**.
- Ideal en **componentes críticos**, bibliotecas o funcionalidades nuevas.

🗨️ ¿Es técnica, metodología, filosofía?

- ✅ **Técnica** de desarrollo.
- 🔄 Con **filosofía** de diseño evolutivo y mejora continua.
- 🧰 Puede formar parte de una **metodología** (p. ej., XP).
- ❌ **No** es un **framework**.

★ Beneficios de aplicar TDD

- **Diseño modular y mantenible.**
- **Feedback instantáneo** ante errores.
- **Refactor seguro.**
- **Documentación viva** (tests explican el comportamiento).
- **Mayor confianza** para modificar código.

✅ Conclusión

TDD es una **técnica disciplinada** que prioriza la calidad desde el inicio y marca un **ritmo cíclico** seguro (Red–Green–Refactor) para crear **código de calidad**, fácil de mantener y alineado con los requerimientos.

03/10

Clase con Meles

Pruebas de Sw/Testing

PUD:

- Requerimientos
- Análisis
- Diseño
- Implementación
- **Prueba**
- Despliegue

Identificar defectos, un testing exitoso es aquel que identifique defectos. El software es una actividad humana intensiva, entonces, errores habrá.

No se puede asegurar calidad con testing. El testing evidencia defectos, cuya presencia se asume.

Desde que aparecen los frameworks ágiles, el agilismo intenta mostrar esta visión del testing, que es una actividad destructiva, que vienen a criticar el trabajo.

testing, actividad intenta buscar defectos, que su presencia se asume, el testing exitoso es el que lo encuentra.

Antes se pensaba que el testing se podía hacer cuando ya teníamos algo más o menos completo para empezar a probar. Esta idea esta basada de que si vos, los requerimientos tienen dos características, ser objetivos, y verificables. Objetivo: no ambiguo (que sea bonito), verificable, demostrar que esa característica se cumple. La característica de verificabilidad engancha a la prueba. si no tenemos una manera para poder mostrar que algo funciona, entonces no sirve...

Los enfoques empiezan a evolucionar a partir de esto, que se trata de adelantar el testing a partir del principio, ejemplo las user stories, relacionado con las 3c, la confirmacion, verificar escenarios de prueba que muestren al user owner que esa user story esta bien...

El testing es muy caro, por eso empresas deciden ni hacer testing, ejemplo Microsoft.

Tiempos/ Esfuerzos

- Requerimientos
- Análisis
- Diseño
- Implementación 33 - 35%
- **Prueba** 30 - 50%
- Despliegue

El primer modelo que se formaliza a veces es la implementación, todo lo demás no... En algunas empresas.

Mientras más exitoso es el proceso de software, a la gente de testing se le hará más difícil.

Hay que invertir en un desarrollo exitoso, para no gastar tanto en las pruebas, en testing :p

Terminología

Error vs defecto

Encuentra defectos, porque los defectos los introduce alguien de una etapa anterior

Cuando una acción se trasladó a una etapa posterior, es un **defecto**

Cuando se encuentra en una etapa, y en esa etapa se corrige, es un **error**

Mientras más viejo es, más costoso corregirlo.

Los errores hay que evitar que se conviertan en defecto

Tipos de errores (creo que algunos)

- Errores de arquitectura
- Errores de requerimiento

Falla

Un mal funcionamiento del sistema. A veces los errores y los defectos producen fallas en el sistema. Lo provoca un error o defecto.

Falla en el sistema

Mal funcionamiento, o que el sistema deje de andar, hay que ver la funcionalidad. Si lo vemos con el sentido más amplio, en términos de satisfacción del cliente, un mal calculo del IVA es una **falla**

Automatización del testing

Los perfiles profesionales de la gente de testing, hace que se haya especializado un poco más. Ya no cualquiera maneja esto, algo de implementación deben saber para usar una herramienta, se ganan un poco de reconocimiento en la industria.

En las empresas tiene la sensación de que si no programas no estas trabajando.

✓ Éxito

Testing es una actividad de control, hago una comparación contra algo, si tengo una versión de producto, la tengo que probar contra algo, contra los requerimientos.

Un error que no se puede reproducir, no es un error

✓ Éxito

Requerimiento es general, caso de prueba específico

Caso de uso tiene sumatoria de escenarios, tenemos una plantilla que describía un escenario en particular. Esto es genérico. Se pide que seleccione país, el ingresa el país, no dice el VALOR. Cuando generamos un *caso de prueba*, es un caso concreto y específico. Basado en los requerimientos. Juan Perez está loguado, selecciona el país argentina, provincia cordoba, damos de alta el barrio nueva cordoba, es todo ESPECÍFICO

Testing metodológico

Planificación y diseño

--- 50%

Ejecución

Seguimiento y cierre

Se documenta el paso a paso de cómo ocurrió el error

Procesos de testing:

- Planificación y diseño
 - Artefacto *Plan de Prueba* y el más importante, *Caso de Prueba*, eso me determina que hago un Testing Metodológico y si lo puedo reproducir. Se muestra un caso de prueba específico de pasos...
 - Caso de Prueba tiene *seteo y escenarios* y *resultado esperado*. El seteo es específico. El caso de prueba es el paso...
 - *Se requiere el nivel de detalle más fino*
- Ejecución
 - Reporte de defectos
 - Requiere lo anterior para hacer los casos de prueba.
 - Evidencia de acción de prueba
- Seguimiento y cierre (Informa la situación de cómo se halló)

Testing AdHoc

Prueba manual?

No se puede reproducir.

No sirve para nada

Sentarme a ver cosas :p

Si pongo letras en la fecha?

Ponele como fuerza bruta

No tengo algo para solventar

Si salta algo, como lo puedo comunicar

Informar paso a paso de como se reproduce

Ciclo de prueba

Un ciclo de ejecución y seguimiento y cierre.

Planificación y diseño no

Ciclo cero

Primer ciclo que se hace

Invalidante

No puedo hacer nada en el sistema

Adelantar

La parte del 50 arriba se puede adelantar, cuando tenemos ya los requerimientos acordados...

Lo de abajo no lo puedo adelantar, lo de arriba si se puede, se debe adelantar...

Adelantamos el plan de prueba, tomamos como entrada los requerimientos y hacemos eso...

Prueba tiene

Validación

Determina adecuación, si se definió , se creó el producto correcto

Verificación

Que funcione correctamente

Determina corrección, si eso que hice, funciona bien

Advertencia

No hay una definición de requerimientos, por eso en muchas empresas es difícil implementar pruebas

Error que se comete mucho cuando no tengo requerimientos, y lo único que tengo es el código, puedo hacer verificación, pero no validación...

Los casos de prueba tienen que estar basados en los requerimientos.

No hay que esperar a que esté terminado el código para probar..

Hay que adelantar el proceso de testing...

Niveles de Prueba

4 niveles de prueba

Se prueba de lo particular a lo general

- Prueba unitaria - lo hace el desarrollador
- Prueba de integración - Prueba de Interfaces - Que ensamble
- Prueba de sistema o versión
- Prueba de aceptación de usuario

Ambientes de prueba / Entorno

- Desarrollo
- Prueba - stage
- Preproducción
- Producción

clase 15-10

Vamos a ver estrategias de testing tradicional en la materia

Se pueden aplicar también en ágil...

Estrategias de testing

- Caja Negra
Tenes entradas y salidas
Es dinámica
Probar ejecutando la aplicación
Ante ciertas entradas, espero ciertas salidas
Es un testing dinámico, ejecuto el software para probarlo, determinar que a unas entradas, las salidas son las esperadas
- Caja Blanca
Sabes el comportamiento interno
Probamos el código, si va por acá hace esto, si va por acá hace lo otro, es estático, no es lo mismo que debug. Es solo mirar, no ejecuto el código, por eso no es debug.

TDD no es una metodología de testing, es una metodología de desarrollo, primero piense en las pruebas, que pruebas, a nivel unitario

Las de caja negra la pensamos como testing de sistemas...

Ver pruebas estáticas y dinámicas

✓ Éxito

Las estrategias tienen como objetivo es conseguir la mayor cobertura de pruebas al menor costo posible, la forma mas eficiente posible. (pregunta final)

En caja blanca, probar las combinaciones de las líneas de código, cubrirlas todas...

Caso de Prueba

Agrupamos los casos de prueba en casos de prueba que supongo que funcionan similar, asumo que los grupos andan de forma similar...

Hay que saber definir como asumo... Por **clases de equivalencia**.

Son clases de grupos de casos de prueba

Veremos

1. Partición de equivalencia
2. Análisis de valores límites

Las dos requieren la definición de clase de equivalencia...

Clases de equivalencia

Hay algunas que están referidas a las entradas y también a las salidas

Hay de válidas e inválidas(?)

Hay de condición extrema(?)

El conjunto tal es inválido, el conjunto tal es válido
Hay salidas esperadas a cada una de estas entradas

	Condicion externa	Valida	Invalida
Entrada	edad	numero mayor o igual a 18	numero menor a 18
Salida	mensaje edad	te pudiste registrar, sos mayor de 18	no te pudiste registrar, sos menor de 18

Un caso de prueba completo es la edad y el cuit, y mandar a registrar. sino dejar vacio.

conjunto minimo de casos de prueba que me haga la mayor cobertura de pruebas.

cuando definimos las clases, escribimos los casos de prueba

me voy y me fijo en cada una de las condiciones externas que elementos puedo sacar de ese pool para armar mi caso de prueba, y tengo que asegurar de cubrir al menos una vez esa clase de equivalencia, cuando cubro todo me dejo de escribir casos de prueba

LEAN

Lean es, ante todo, **una filosofía/mentalidad de gestión** enfocada en **maximizar valor para el cliente eliminando desperdicio** y aprendiendo rápido. En software, Mary y Tom Poppendieck traducen esos principios de Lean manufacturing a prácticas de desarrollo: ver el desperdicio, mapear la cadena de valor y optimizar el flujo de punta a punta, no partes aisladas.

Los 7 principios Lean (versión software) y su sentido práctico

1. **Eliminar desperdicio**: todo lo que no agrega valor percibido por el cliente es desperdicio —hay que aprender a verlo.
2. **Ampliar el aprendizaje**: desarrollo = descubrimiento; necesitamos ciclos cortos de diseño-implementación-feedback.
3. **Tomar decisiones lo más tarde posible**: mantener opciones abiertas en entornos inciertos produce decisiones mejores (basadas en hechos).
4. **Entregar lo antes posible**: la velocidad corta ciclos de descubrimiento, habilita feedback temprano y evita construir lo que ya quedó viejo.
5. **Dar poder al equipo (empower)**: quienes hacen el trabajo deciden mejor; estándares existen para ser mejorados por la gente.

6. **Embeber la integridad conceptual**: coherencia del sistema (arquitectura, usabilidad, evolución) como cualidad construida desde adentro.
7. **Ver el todo**: evitar suboptimizar partes; optimizamos el sistema end-to-end.

Lean Software Development - An ...

Los autores los presentan como “siete reglas simples” que guían decisiones en contextos cambiantes.

“Gastos” o desperdicios en producción Lean (muda)

Lean identifica **siete desperdicios** de manufactura que en software se traducen así:

Inventario → **trabajo parcialmente hecho**,

Sobreprocesamiento → **procesos extra**,

Sobreproducción → **features extra**,

Transporte → **task switching/traspasos**,

Espera → **espera**,

Movimiento → **movimiento**,

Defectos → **defectos**.

Dos tipos de desperdicio

- **Tipo 1 (necesario por ahora)**: no agrega valor al cliente pero hoy es inevitable (p. ej., cierta documentación regulatoria). Minimizar.
- **Tipo 2 (puro desperdicio)**: no agrega valor ni es obligatorio. Eliminar.

Los **7 + 1** desperdicios (definición breve y ejemplos en software)

1. **Sobreproducción**: construir **más** o **antes** de lo necesario (features “por las dudas”). Provoca acumulación y re-trabajo.
2. **Inventario**: **trabajo parcialmente hecho** (requisitos, código o casos de prueba no integrados). Envejece, oculta problemas y bloquea capital.
3. **Movimiento**: desplazamiento innecesario de personas/artefactos (p. ej., pedir info de un área a otra sin flujo claro).
4. **Espera**: colas para aprobación, pruebas o ambientes; baja el flujo y alarga el lead time.
5. **Sobreprocesamiento**: **procesos/entregables extra** que no aumentan valor (papelería por papelería). Si hace falta, mantenerlo corto, alto nivel y offline.
6. **Defectos**: errores detectados tarde multiplican desperdicio; solución: probar de inmediato, integrar seguido y liberar pronto.
7. **Transportación** (traspasos)/Task switching: mover trabajo/artefactos entre manos con pérdida de conocimiento; el multitasking agrega demoras. **+1) Talento no utilizado**: no aprovechar habilidades/criterio de la gente (lo contrario a “empower

the team” y “respeto por las personas”). Lean remarca desarrollar personas que mejoran el proceso.

Filosofía KANBAN

Kanban es un método para definir, gestionar y mejorar servicios (trabajo del conocimiento) aplicando un enfoque holístico orientado al cliente y al flujo. Visualiza el trabajo, limita el WIP y mejora la entrega con cambios evolutivos, sobre un proceso ya existente (no reemplaza marcos como Scrum: se “añade” al modo de trabajar actual).

En **japonés**, “**kanban**” significa *señal*, *tarjeta de señalización*, *letrero* o *tablero visual*; Toyota llamó “sistemas kanban” a los mecanismos que **limitan el trabajo en progreso** mediante señales (tarjetas físicas o virtuales).

Kanban bebe del **Lean/TPS**. En la práctica, **Just-in-Time** introduce la planificación *pull* usando **kanban**; el mecanismo se inspiró en cómo los **supermercados** reponen estantes: el consumo aguas abajo *tira* de la reposición aguas arriba.

Valores de Kanban

Los **nueve valores** que orientan comportamientos y prácticas son: **transparencia, equilibrio, colaboración, orientación al cliente, flujo, liderazgo, comprensión, acuerdo y respeto**.

Principios de Kanban

1) Principios de **Gestión del Cambio**

- **Empezar con lo que haces ahora** (entender procesos actuales y **respetar roles y responsabilidades**).
- **Acordar buscar la mejora mediante cambio evolutivo**.
- **Fomentar liderazgo a todos los niveles**.

2) Principios de **Entrega del Servicio**

- **Comprender y enfocarse** en necesidades/expectativas del cliente.
- **Gestionar el trabajo** y dejar que las personas **se auto-organicen** alrededor de él.
- **Revisar periódicamente** la **red de servicios** y sus **políticas**.

Prácticas generales (cómo se aplica)

- **Visualizar** el trabajo/proceso.
- **Limitar el WIP**.

- Gestionar el flujo.
- Hacer explícitas las políticas.
- Ciclos de retroalimentación.
- Mejorar colaborativamente y evolucionar experimentalmente.

¿Cómo funciona un **sistema kanban**?

Pones en circulación un número de **tarjetas** (capacidad acordada). **Sólo** puedes iniciar nuevo trabajo cuando **queda una tarjeta libre** (señal). Al terminar un ítem, **reciclas** la tarjeta y habilitas el siguiente. Es un **sistema pull**: **no se sobrecarga** si el límite está bien definido.

¿Para qué sirve Kanban?

- **Recuperar control y aliviar sobrecarga** limitando WIP;
- **Hacer visibles cuellos de botella, variabilidad y defectos** para atacarlos;
- **Mejorar flujo, previsibilidad y lead time**;
- **Generar confianza** con entregas cadenciadas.

¿Dónde se usa?

Más allá de TI, hay muchos ejemplos en **marketing, RR.HH., diseño/medios, atención al cliente, desarrollo de productos y educación**. Kanban es un “**método sin metodología**” con **amplio rango de aplicación**, siempre **sobre** flujos existentes.

¿Es una filosofía?

Formalmente, **Kanban es un método de gestión** con **principios y valores**; en la práctica, muchas organizaciones lo viven también como **filosofía de mejora continua y orientación al servicio**. Pero no es una **metodología** ni un **framework** prescriptivo.

Resumen en una frase

Kanban = *gestionar servicios visualizando el trabajo, limitando WIP y mejorando el flujo con cambios evolutivos, guiados por valores y principios que ponen al cliente y al sistema por delante de las tareas individuales.*

Qué es WIP (y por qué NO es “Working”)

WIP = Work In Progress = Trabajo en curso. Es **la cantidad de elementos** que hay dentro del sistema (o de una parte) **en este momento**. Es una de las **tres métricas clave** de Kanban junto con **Lead Time** y **Throughput** (tasa de entrega).

Por qué **limitar** el WIP mejora el flujo

- Si “llenás” el sistema al **100% de ocupación**, el rendimiento **se desploma**: como una autopista en hora pico; hay **mucho trabajo dentro** pero **poco sale por unidad de tiempo** y los tiempos de entrega se disparan. Kanban recomienda trabajar **por debajo de la capacidad máxima** (holgura) para que **el flujo sea suave y predecible**.

Principios de Kanban		
Gestión de Cambios	Comenzar con lo que haces ahora	Entender los procesos actuales tal y como están siendo realizados en la actualidad, Respetar los roles actuales, las responsabilidades de cada persona y los puestos de trabajo.
	Acordar la búsqueda de la mejora a través del cambio evolutivo	
	Fomentar actos de liderazgo a todos los niveles	
Entrega de Servicios	Comprender y enfocarse en cumplir las necesidades y expectativas del cliente	
	Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él	
	Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados	



1) Visualizar

Qué visualizar en un tablero (no solo “todo lo que hay que hacer”):

- Flujo de trabajo real** (de izquierda a derecha; del **punto de compromiso** a **entrega**), no un ideal.
- Tipos de trabajo / carriles** (clases de servicio, proyectos, etc.).
- Límites de WIP** por columna/estado/equipo.
- Riesgos y bloqueos** (marcarlos para que salten a la vista).

- **Punto de compromiso y de entrega** claramente señalados.

La visualización permite **absorber mucha info rápido**, alinear entendimiento y **decidir mejor**.

2) Limitar el trabajo en curso (WIP)

- Elegí **dónde** limitar (por estado, carril, persona o sistema). Empezá por las columnas de “En curso” y ajustá.
- Un límite simple (ej.: “2 en desarrollo”) ya cambia el juego.
- No es “trabajar menos”, es **acabar más** y **mejor** (menos multitarea, más colaboración, calidad).

3) Gestionar el flujo

- Objetivo: **terminar** trabajo **fluida y predecible** a un **ritmo sostenible**. Se apoya en WIP limitado y **medir** para ajustar.
- Mirá métricas simples (lead time, throughput, WIP) y su **distribución** para reducir variabilidad y mejorar puntualidad.

El objetivo es poder terminar el trabajo de la forma más fluida y predecible posible, mientras se mantiene un ritmo sostenido. Limitar el trabajo en curso nos ayuda a asegurar un flujo suave y predictivo. El seguimiento y la medición del flujo de trabajo da como resultado información valiosa y útil para gestionar las expectativas de los clientes, hacer predicciones y mejorar.



4) Hacer las políticas explícitas

- Escribí y hacé visibles reglas como: **cuándo reponer** el backlog, **criterios de “Terminado”**, **clases de servicio**, **agenda de cadencias**, **quién decide qué**.
- Deben ser **pocas, simples, bien definidas, visibles, siempre aplicables y fáciles de cambiar** por quienes hacen el trabajo.
- Las políticas (incluido el WIP) son **restricciones habilitantes**: guían la auto-organización y permiten **experimentar**.

5) Establecer ciclos de retroalimentación (cadencias)

- Reuniones cortas y regulares para **pull de trabajo**, **revisión del flujo**, **replenishment**, **delivery review**, **retro**.
- Las cadencias pueden estar **desacopladas** (no hace falta iterar todo junto).

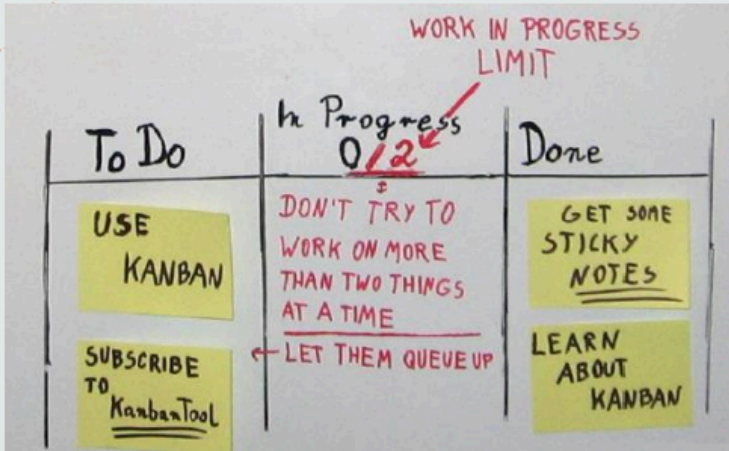
6) Mejorar colaborativamente y evolucionar experimentalmente

- Cambios **evolutivos, guiados por datos** y **experimentos seguros-de-fallar**; Kanban es empírico: probás, medís, ajustás.

“Limitar el trabajo en progreso... ¿no es empeorar?”

Al revés. **Menos cosas abiertas a la vez** ⇒ **más cosas terminadas antes**, con mejor calidad y previsibilidad. Eso **desbloquea capacidad** para ayudar al cuello de botella, reduce esperas y hace que el sistema se comporte **mejor que a 100% de ocupación**. Pensá en la autopista: un poco de **holgura** evita la parálisis.

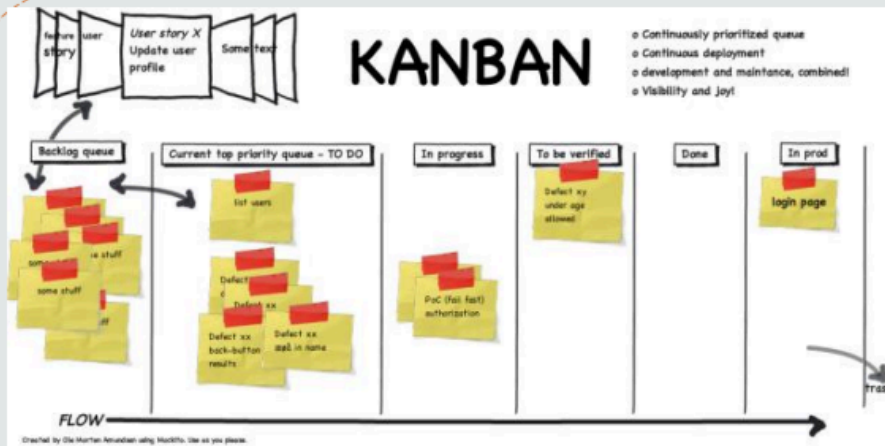
Límite WIP



+ Asignar límites explícitos de cuántos ítems puede haber en progreso en cada estado del flujo de trabajo.

Administración de Colas

Foco es en Flujo “fin a fin” FLOW = Centrado en el Cliente



¿Para qué sirve “modelar el proceso” en Kanban?

Modelar = **dibujar tu flujo real de trabajo** (no el ideal) desde el **punto de compromiso** (cuando aceptas empezar algo) hasta la **entrega**. Se representa en un **tablero** con columnas (estados) y, si hace falta, carriles por tipo de trabajo. Esto permite **ver cuellos de botella**, **limitar WIP** y **gestionar el flujo** con datos.

Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En Producción
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	

- **Cola de Producto (Backlog):** opciones aún no comprometidas.
 - **Análisis (En progreso/Hecho) → Desarrollo (En progreso/Hecho):** trabajo que atraviesa etapas intermedias.
 - **Listo para Build:** buffer que protege a la etapa siguiente.
 - **En Testing (En progreso/ Listo para Despliegue).**
 - **En Producción:** entregado al cliente (punto de salida).
- Este patrón (izquierda→derecha, con buffers) es típico y se adapta al flujo real del servicio.

¿Cómo aplicar Kanban? (pasos mínimos y prácticos)

Un arranque recomendado es **STATIK** (enfoque sistémico para introducir Kanban).
Resumen práctico:

1. **Entiende la demanda y el propósito del servicio** (quién pide qué y por qué).

2. **Modela el flujo actual** en un tablero.
3. **Visualiza riesgos/bloqueos** y **haz políticas explícitas**.
4. **Pon límites de WIP** en una o dos columnas clave.
5. **Empieza con cadencias** (reuniones cortas) y **mide** (Lead time, Throughput, WIP).
6. **Mejora por experimentos** (cambios pequeños, medidos).

STATIK no es lineal: **iterás** y vas ajustando.

“Dividir el trabajo en piezas” y **tipos de trabajo**

Usás tarjetas para **requerimientos**, **defectos**, **mantenimiento/refactor** y **solicitudes** (soporte, mejoras, etc.). Podés asignar **colores** y **capacidad por tipo** (p. ej., 60% features, 30% mantenimiento, 10% defectos) para alinear oferta con demanda y evitar que un tipo “ahogue” a los demás.



Hacer **políticas explícitas** (la lámina de “acuerdos”)

Las políticas son **reglas visibles** que todos entienden y usan:

- **Cuándo y quién repone** el tablero (replenishment).
- **Criterios para mover** una tarjeta (“listo para test”, “definición de terminado”).
- **Límites de WIP** y **asignación de capacidad**.
- **Clases de servicio** (cómo se prioriza).
- **Agenda de cadencias** (diaria, revisión de entrega, etc.).

Deben ser **pocas, simples, visibles, aplicables siempre y fáciles de cambiar** por quienes dan el servicio. Son **restricciones habilitantes** que fomentan foco y

colaboración.

Hacer explícitas las políticas

- + Todas las políticas deben ser acordadas entre todas las partes involucradas, incluyendo a los clientes, interesados y trabajadores responsables del trabajo que está en el tablero.
- + Las políticas deben estar expuestas en un lugar destacado.
- + A nivel de equipo, los acuerdos son una buena forma de introducir las políticas.



Definir tipos de trabajo...

Asignando capacidad en función de la demanda



Requerimientos

Caso de uso
Historias de Usuario
Porciones de Casos de Uso
Características



Defectos

Defectos en Producción
Defectos



Desarrollo

Mantenimiento
Refactorización
Actualización de Infraestructura



Solicitudes

Solicitud de Cambio
Sugerencias de Mejora

Políticas de calidad: qué definir y por qué

En Kanban las **políticas** son reglas visibles, acordadas y aplicables por todos. Hacerlas explícitas es práctica central: “hacer las políticas explícitas” está entre las seis prácticas generales del método.

Qué conviene definir:

- **Reposición/entrada de trabajo** (replenishment): quién prioriza, cada cuánto y cuánta “nueva” demanda se toma. Esto suele materializarse en la **Team Replenishment Meeting** y políticas de priorización (p. ej., FIFO, por clase de servicio).

- **Definición de “Terminado” y criterios de transición** (“qué tiene que cumplirse para mover una tarjeta”): pruebas pasadas, code review hecho, documentación mínima, etc. Estas políticas se usan para “arrastrar” (pull) el ítem a la siguiente columna.
- **Límites de trabajo en curso (WIP limits)** por columna o carril: fijan cuántos ítems pueden estar “a la vez” en cada estado. Sin límites no hay sistema Kanban efectivo.
- **Clases de servicio** (cómo se trata cada tipo de demanda y con qué promesa de tiempo): estándar, fecha fija, expedite/exprés, intangible... y sus **SLA/objetivos de tiempo de entrega**.
- **Cadencia de reuniones** (ritmo y propósito): dailies de flujo, reposición, retrospectivas, revisiones... Definir frecuencia y agenda aumenta la agilidad.

Cadencias: qué reuniones, cada cuánto y para qué

La Guía Oficial recomienda empezar simple y evolucionar la **red de cadencias** según contexto. Tres básicas a nivel de equipo:

Cadencia	Frecuencia típica	Propósito
Team Kanban Meeting	Diaria	Observar el flujo (no a las personas). ¿Qué desbloquear? ¿Qué tomar después?
Team Retrospective	Quincenal/mensual	Reflexionar sobre cómo se gestiona el trabajo y mejorar.
Team Replenishment	Semanal o a demanda	Seleccionar (reponer) los próximos ítems desde la lista de trabajo.

Métricas clave: definiciones, uso y diferencias

Las métricas no son para “controlar personas”, sino para **entender y mejorar el sistema**.

Lead Time (Tiempo de entrega)

Tiempo desde el **punto de compromiso** (cuando aceptas hacer algo) hasta que está **terminado/entregado**. Es la mirada del cliente (¿cuánto tarda en llegar lo que pidió?). Úsalo para promesas y SLAs.

Cycle Time (Tiempo de ciclo)

Tiempo desde que el ítem **entra efectivamente al proceso de trabajo** (p. ej., a la primera columna “en curso”) hasta que termina. Es mirada interna de ejecución.

Touch Time (Tiempo tocado)

Suma de los **periodos de trabajo activo** sobre el ítem (excluye esperas/colas).

Siempre cumple:

Touch Time $\leq$ Cycle Time $\leq$ Lead Time.

Sirve para detectar cuánta parte del tiempo total es **valor agregado** vs. espera. La **eficiencia de flujo** suele aproximarse con Touch Time / Lead (o / tiempo transcurrido).

Otros básicos que acompañan

- **WIP**: cuántos ítems hay “vivos” en el sistema.
- **Tasa de entrega (throughput)**: ítems terminados por unidad de tiempo.
- **Gráficos útiles**: gráfico de ejecución (run chart) de lead time, histograma de distribución y diagrama de flujo acumulado (CFD) para ver colas y estabilidad.

Cuándo usar cada una?

- Promesas y acuerdos con cliente → **Lead Time / SLA por clase de servicio**.
- Capacidad interna, cuellos de botella → **Cycle Time** y **WIP**.
- Eficiencia y despilfarro (mucho espera) → **Touch Time** y la relación TT/Lead.

Métricas orientadas al servicio (para tus diapositivas)

Cuando Kanban se aplica “como gestión de servicios”, medimos la **promesa y la capacidad de cumplirla**:

- **Expectativa de nivel de servicio** (lo que el cliente espera).
- **Capacidad real del sistema** (lo que podemos entregar con variabilidad conocida).
- **Acuerdo de nivel de servicio (SLA)** por clase de servicio (p. ej., “estándar: 85 % en $\leq$ 10 días”, “expedite: 95 % en $\leq$ 2 días”).
- **Umbral de adecuación** (por debajo de cierto performance es inaceptable y hay que actuar). Esta lógica está alineada con la guía al definir clases de servicio, SLAs y usar métricas de lead time y distribución para asegurar **predictibilidad**.

Consejos prácticos para bajar esto a tu tablero

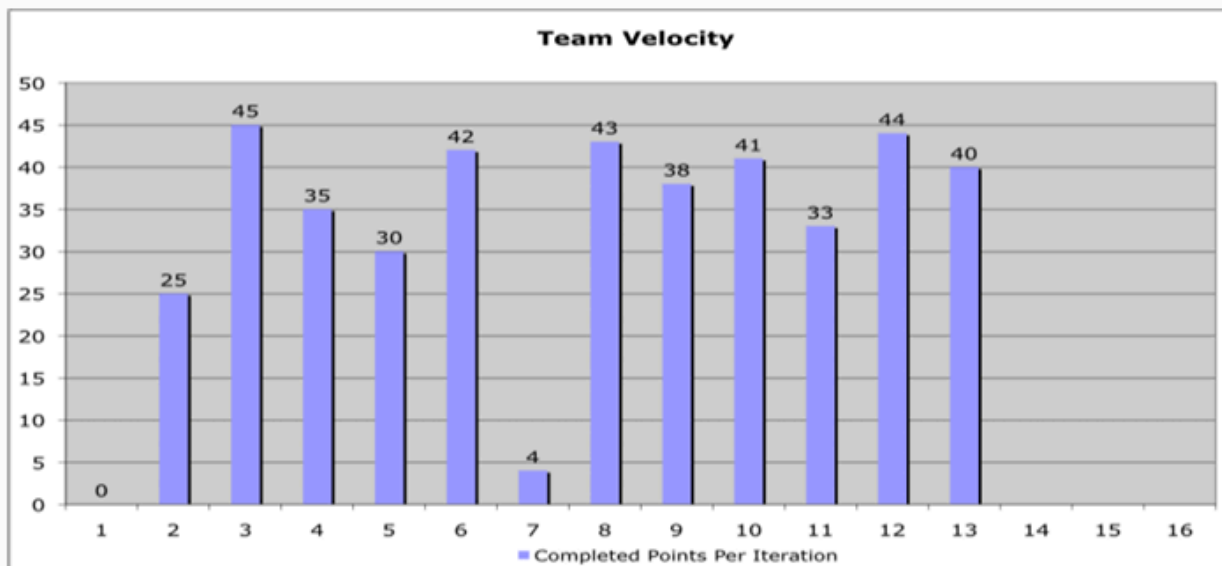
1. **Escribe 5–10 políticas visibles** al lado del tablero (reposición, “Definición de Terminado”, límites WIP, clases de servicio y sus SLAs, dailies y reposición: frecuencia y agenda).
2. **Arranca con tres cadencias**: daily de flujo (15 min), reposición semanal, retro mensual. Ajusta ritmos con la experiencia.
3. **Mide desde el día 1**: fecha de compromiso y de “done” en cada tarjeta → ya tienes **lead time**; registra columnas de entrada/salida para **cycle time**; cronometra el trabajo activo si quieres **touch time**. Usa un run chart y un CFD sencillos.

4. **Usa límites WIP pequeños** para forzar conversación y mejora del flujo (si se bloquea algo, todos ven el atasco y ayudan a desbloquear).

Métricas

Falta-

Velocidad, cantidad de storypoints que vamos quemando, definimos los storypoints en la review? en la demo. La idea es que esta velocidad sea estable, en los primeros sprints es 0. se debería regularizar la velocidad

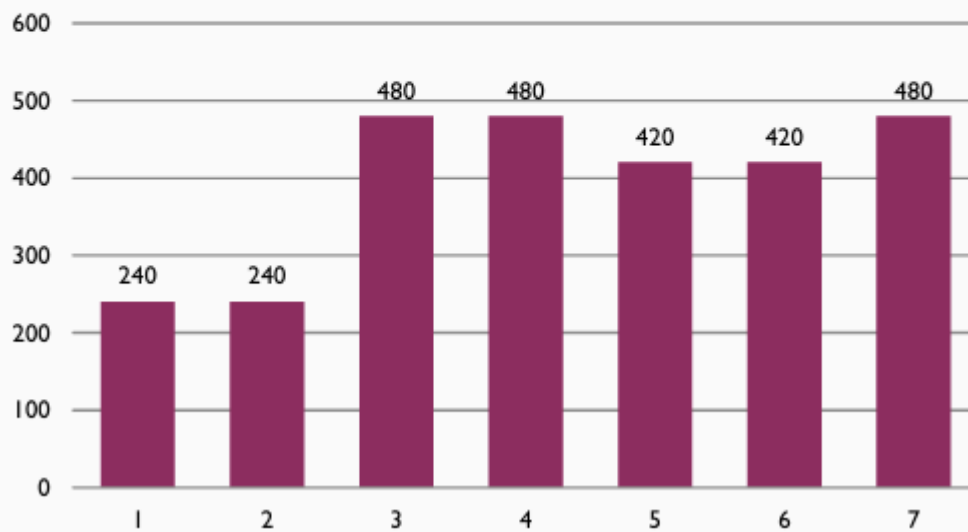


Capacidad, cuanto trabajo le va a tomar al equipo hacer el proximo trabajo, estimo en el mejor de los casos, cuantos puntos de historia voy a ser capaz de quemar en el proximo sprint. Usamos la descomposicion de tareas del backlog. se hace en la

planing

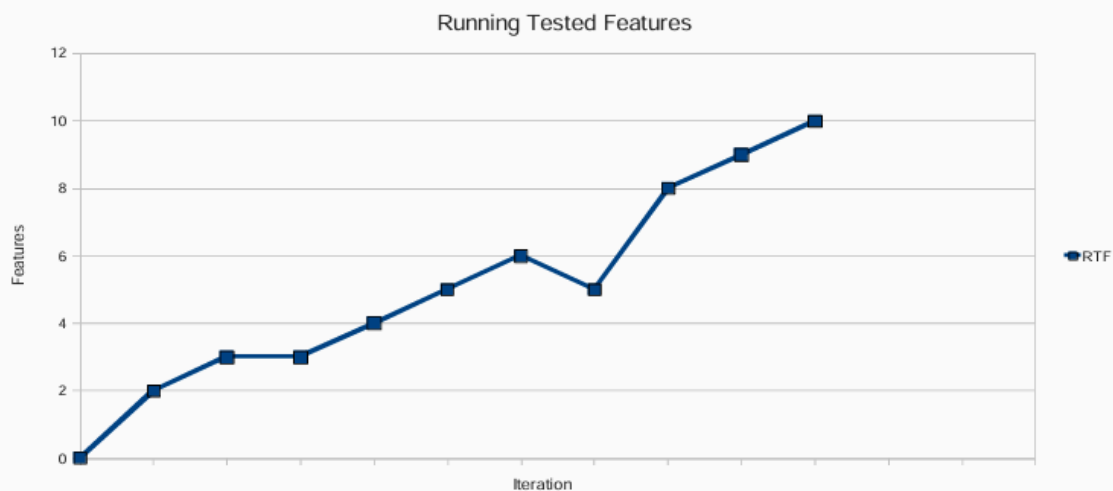
Sprint	1	2	3	4	5	6	7	Total
Horas	240	240	480	480	420	420	480	2760
Puntos de Historia	30	30	45	60	58	52	60	335

Capacidad



No se usa demasiado, **RTF**, es la ultima de scrum, vemos una medida de proceso, pero no nos dice la cantidad de software que entregamos? Se calcula el rtf en la review, mismo momento que el velocity

RUNNING TESTED FEATURES (RTF)



La profe pide mucho saber en que etapa de scrum van estas metricas

Métricas en kanban

Kamban vs scrum, scrum gestion de proyecto y kanban al flujo de trabajo, gestion de mejora de procesos, pero procesos continuos, no en la gestionn d eproyectos.. Usa enfoque lean, tiene cosas en comun con scrum. scrum tiene muchas cosas definidas, los roles, como trabajar, en la cual kanban no va tanto al detalle.

Métricas en kanban, **Lead time** tiempo que tarda desde pedir una característica hasta que se recibe, **cycle time** desde que se empieza a hacer el trabajo, hasta que terminamos de hacer la pieza de trabajo

Touch time, tiempo que efectivamente se labura con la pieza de trabajo

Definir un sla con el cliente cual de las metricas utilizar: lead time. definir el sla con el cliente.

Cuanto mas cerca el lead time y el touch time, mejor

KANBAN: MÉTRICAS CLAVE

Cycle Time (Tiempo de ciclo)

- Es la métrica que registra el tiempo que sucede entre el inicio y el final del proceso, para un ítem de trabajo dado. Se suele medir en días de trabajo o esfuerzo.
- Medición más mecánica de la capacidad del proceso
- **Ritmo de Terminación**

Lead Time (Tiempo de entrega)

- Es la métrica que registra el tiempo que sucede entre el momento en el cual se está pidiendo un ítem de trabajo y el momento de su entrega (el final del proceso). Se suele medir en días de trabajo.
- **Ritmo de entrega**

KANBAN: MÉTRICAS CLAVE

Touch Time (Tiempo de Tocado)

- El tiempo en el cual un ítem de trabajo fue realmente trabajado (o "tocado") por el equipo.
- Cuántos días hábiles pasó este ítem en columnas de "trabajo en curso", en oposición con columnas de cola / buffer y estado bloqueado o sin trabajo del equipo sobre el mismo.

$$\textit{Touch Time} \leq \textit{Cycle Time} \leq \textit{Lead Time}$$

Eficiencia del Ciclo de Proceso

$$\% \text{ Eficiencia ciclo proceso} = \textit{Touch Time} / \textit{Elapsed Time}.$$

18

Metricas en kanban

Kanban nos sirve para la gestion de cualquier serrvivio orientado al conocimiento, no solo para la construccion de software, poder acordar con el cliente cual es el servicio que quiere, que nivel de servicio quiere. es muy diferente a crum. quesolo mir el producto. aca es orientada al servicio

RESUMIENDO MÉTRICAS EN CADA ENFOQUE

■ Tradicionales

- Esfuerzo
- Tiempo
- Costos
- Defectos

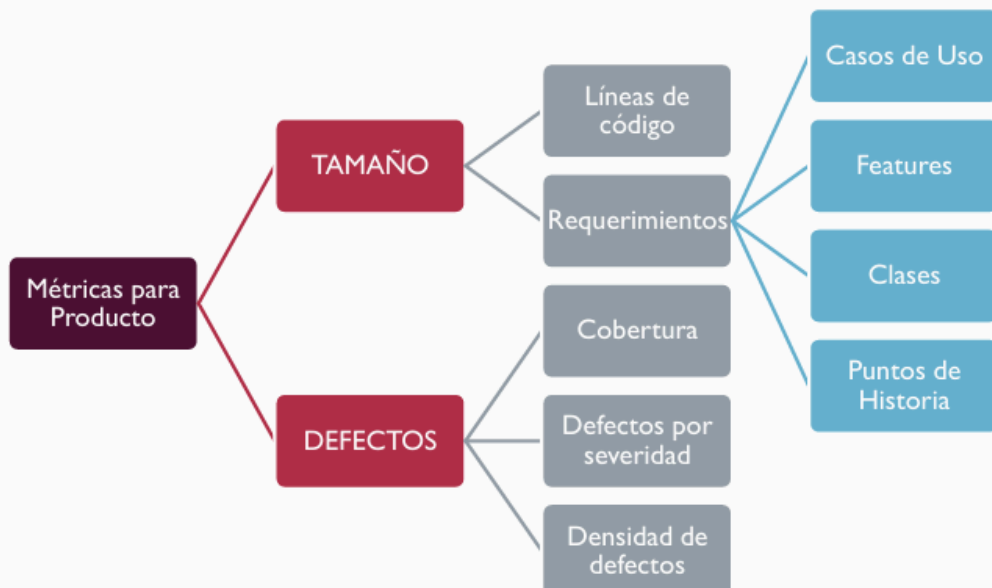
■ Ágiles

- Velocidad
- Capacidad
- Running Tested Features

■ Lean

- Lead Time
- Cycle Time
- Touch Time
- Eficiencia Proceso

¿Y PARA EL PRODUCTO SOFTWARE, QUÉ MÉTRICAS ?



no hay una que funcione para todo, no hay una receta.. o importante es las métricas **no miden para medir personas, productividad, nos sirve para mejorar la forma que trabajamos, no importa el framework**